Lab 6: Spring Mail & sending emails

Send transactional HTML notifications asynchronously using JavaMailSender and Thymeleaf

Continue project from Lab 5.

Not cover in this lab: file attachments, CSV batch exports, third-party cloud SDKs (SendGrid / AWS SES).

What you will learn

- Add spring-boot-starter-mail dependency to the pom.xml file.
- Configure SMTP server connection properties in application.properties (using Mailtrap sandbox).
- Enable asynchronous method execution with @EnableAsync to prevent SMTP delays from blocking HTTP threads.
- Create a branded HTML email template using Thymeleaf in the templates/mail/todo-notification.html file.
- Implement EmailService using JavaMailSender, MimeMessageHelper and TemplateEngine with @Async.
- Trigger email notifications automatically whenever a new todo is created via HTML or RESTful API.

Responsiveness with Asynchronous Mail Delivery

Connecting to an SMTP server and delivering an email takes between 1 and 3 seconds. By annotating the sending method with @Async, the email dispatch executes in a background thread, allowing the HTTP controller to respond to the browser immediately without any latency.

✓ Step 1. Add Spring Boot Starter Mail dependency to the pom.xml file

Open the pom.xml file and add spring-boot-starter-mail inside the <dependencies> block.

This provides JavaMailSender and Jakarta Mail APIs.

pom.xml (add inside <dependencies>)

```
<!-- Spring Boot Starter Mail: Jakarta Mail + JavaMailSender -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-mail</artifactId>
</dependency>
```

- spring-boot-starter-mail: auto-configures a JavaMailSender bean based on application properties.
- Reload Maven dependencies in your IDE after updating the pom.xml file.

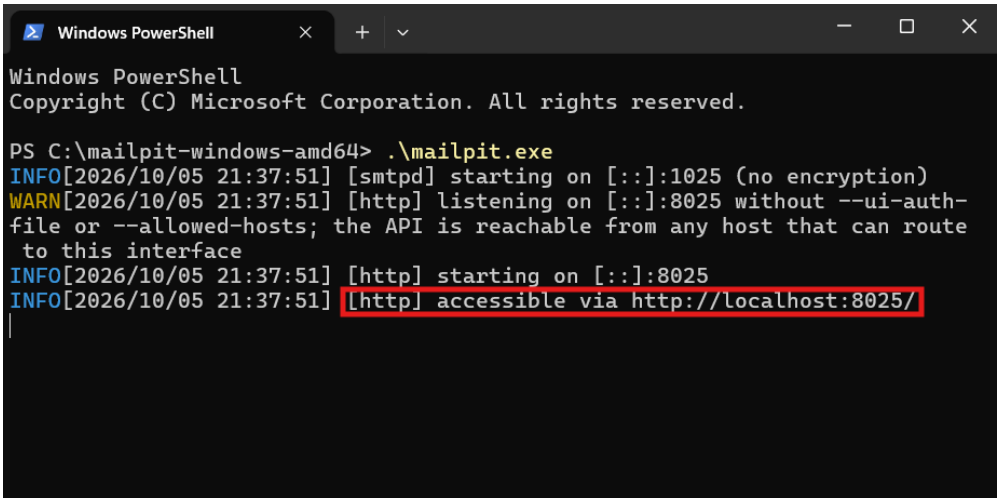
✓ Step 2. Configure SMTP Server in application.properties

We will use the Mailpit application to act as a local mock SMTP server for Spring Boot application instead of sending real emails.

- Official web page: https://mailpit.axllent.org/
- Download page: https://github.com/axllent/mailpit/releases
- Download and extract the .zip file to a folder of your choice (e.g., C:\mailpit-windows-amd64)

Open Command Prompt or PowerShell, navigate to that directory and start Mailpit by using these commands

```
cd C:\mailpit-windows-amd64
.\mailpit.exe
```



Then open src/main/resources/application.properties and add the SMTP configuration.

src/main/resources/application.properties (append mail settings)

```
spring.mail.host=localhost
spring.mail.port=1025
# Mailpit does not require a username or password by default
spring.mail.username=
spring.mail.password=
spring.mail.properties.mail.smtp.auth=false
spring.mail.properties.mail.smtp.starttls.enable=false
```

- Mailpit listens for SMTP traffic on port 1025 by default and requires no authentication.

✓ Step 3. Enable asynchronous execution (@EnableAsync)

Update TodolistApplication.java by adding the @EnableAsync annotation.

This tells Spring to scan for @Async methods and run them in background worker threads.

src/main/java/com/example/todolist/TodolistApplication.java

```
package com.example.todolist;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.scheduling.annotation.EnableAsync;

@SpringBootApplication
@EnableAsync
public class TodolistApplication {
    public static void main(String[] args) {
        SpringApplication.run(TodolistApplication.class, args);
    }
}
```

- @EnableAsync: enables Spring's asynchronous method execution capability.
- Methods marked with @Async will run in Spring's default background ThreadPoolTaskExecutor.

✓ Step 4. Project structure overview

Review your updated project layout.

We add service/EmailService.java and templates/mail/todo-notification.html. All other layers remain untouched.

Project package structure

```
src/main/java/com/example/todolist/
├── aspect/
├── config/
├── controller/
├── dto/
├── exception/
├── model/
├── repository/
├── service/
│   └── CustomUserDetailsService.java
│   ├── EmailService.java           # new asynchronous mail delivery service
│   └── TodoService.java            # calls EmailService on todo creation
└── TodolistApplication.java        # marked with @EnableAsync
src/main/resources/templates/
├── mail/                           # HTML email templates
│   └── todo-notification.html      # Thymeleaf email template
└── todos/
```

✓ Step 5. Create HTML email template with Thymeleaf

Create directory src/main/resources/templates/mail and add todo-notification.html.

We use inline styles for high compatibility across email clients.

src/main/resources/templates/mail/todo-notification.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8"/>
    <style>
        body { font-family: Segoe UI, Arial, sans-serif; color: #333; margin: 0; padding:
20px; background-color: #f7f9fa; }
        .card { background: #ffffff; border: 1px solid #dcdcdc; border-radius: 6px; padding:
24px; max-width: 520px; margin: 0 auto; }
        .header { color: #156082; margin-top: 0; font-size: 20px; border-bottom: 2px solid
#156082; padding-bottom: 8px; }
        .field { margin: 12px 0; font-size: 14px; }
        .badge { display: inline-block; padding: 4px 10px; border-radius: 4px; font-weight:
600; font-size: 12px; background: #e8f4fc; color: #156082; }
    </style>
</head>
<body>
<div class="card">
    <h2 class="header">New Todo Notification</h2>
    <p>A new task has been registered in the Todo List system:</p>
    <div class="field"><strong>Title:</strong> <span th:text="${todo.title}">Sample
Task</span></div>
    <div class="field"><strong>Description:</strong> <span
th:text="${todo.description}">Description</span></div>
    <div class="field"><strong>Due Date:</strong> <span th:text="${todo.dueDate}">2026-09-
30</span></div>
    <div class="field"><strong>Status:</strong> <span class="badge"
th:text="${todo.completed} ? 'Completed' : 'Pending'">Pending</span></div>
    <hr style="border: none; border-top: 1px solid #eee; margin-top: 24px;"/>
    <p style="font-size: 12px; color: #777; margin: 0;">Automated notification from Web
Programming (WEPR330479).</p>
</div>
</body>
</html>
```

- th:text="${todo.title}": Thymeleaf dynamically replaces placeholders with fields from the passed Todo entity.
- Inline and basic CSS ensures the email displays properly in Gmail, Outlook and mobile apps.
- Template is stored under templates/mail/ and resolved via SpringTemplateEngine.

✓ Step 6. Create asynchronous EmailService

Create service/EmailService.java.

Inject JavaMailSender and TemplateEngine, render the Thymeleaf template and send the MimeMessage asynchronously.

src/main/java/com/example/todolist/service/EmailService.java

```
package com.example.todolist.service;

import com.example.todolist.model.Todo;
import jakarta.mail.MessagingException;
import jakarta.mail.internet.MimeMessage;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.mail.javamail.JavaMailSender;
import org.springframework.mail.javamail.MimeMessageHelper;
import org.springframework.scheduling.annotation.Async;
import org.springframework.stereotype.Service;
import org.thymeleaf.TemplateEngine;
import org.thymeleaf.context.Context;

@Service
public class EmailService {

    private static final Logger log = LoggerFactory.getLogger(EmailService.class);

    private final JavaMailSender mailSender;
    private final TemplateEngine templateEngine;

    public EmailService(JavaMailSender mailSender, TemplateEngine templateEngine) {
        this.mailSender = mailSender;
        this.templateEngine = templateEngine;
    }

    @Async
    public void sendTodoNotificationEmail(String recipient, Todo todo) {
        try {
            MimeMessage message = mailSender.createMimeMessage();
            MimeMessageHelper helper = new MimeMessageHelper(message, true, "UTF-8");

            Context context = new Context();
            context.setVariable("todo", todo);
            String htmlContent = templateEngine.process("mail/todo-notification", context);

            helper.setTo(recipient);
            helper.setSubject("New Todo Created: " + todo.getTitle());
            helper.setText(htmlContent, true);

            mailSender.send(message);
            log.info("[EMAIL ASYNC] Notification email successfully sent to {}", recipient);
        } catch (MessagingException e) {
            log.error("[EMAIL ERROR] Failed to send email to {}: {}", recipient,
e.getMessage());
        }
    }
}
```

- @Async: runs the entire method in a separate worker thread so the web controller returns instantly.
- TemplateEngine.process(): merges model attributes into mail/todo-notification.html to produce the HTML string.
- helper.setText(htmlContent, true): the boolean flag true instructs JavaMail to send the body as text/html.
- mailSender.send(message): dispatches the email to the configured SMTP server over TLS.
- Logging is kept inside EmailService because @Async methods run on a separate thread, so AOP-based logging cannot reliably capture success or failure here.

✓ Step 7. Trigger email notification in TodoService

Update TodoService.java to inject EmailService and dispatch the notification whenever a new todo is created.

src/main/java/com/example/todolist/service/TodoService.java (snippet)

```
    private final TodoRepository todoRepository;
    private final EmailService emailService;

    public TodoService(TodoRepository todoRepository, EmailService emailService) {
        this.todoRepository = todoRepository;
        this.emailService = emailService;
    }

    public Todo create(TodoRequest req) {
        Todo todo = new Todo(
                req.getTitle(),
                req.getDescription(),
                req.getDueDate(),
                req.isCompleted()
        );
        Todo savedTodo = todoRepository.save(todo);

        // Asynchronously dispatch email notification
        emailService.sendTodoNotificationEmail("student@example.com", savedTodo);

        return savedTodo;
    }
```

- Constructor injection: Spring automatically injects the EmailService bean.
- emailService.sendTodoNotificationEmail(): called immediately after todoRepository.save(todo).
- Because EmailService is @Async, this call returns immediately without delaying the response to the user.

✓ Step 8. Test email delivery via web browser and RESTful API

Run the application and verify that creating a todo triggers an asynchronous email delivered to your Mailtrap inbox.

1. Open Mailpit in your browser by go to this URL http://localhost:8025/ and navigate to your virtual inbox.
2. Start TodolistApplication.java in your IDE.
3. Open http://localhost:8080/todos, log in as 'user' / 'password123' and click 'New todo'.
4. Fill out the form: Title = 'Complete Lab 6', click Save: notice the web page redirects IMMEDIATELY with zero lag!
5. Check console logs: notice [EMAIL ASYNC] Notification email successfully sent to student@example.com.
6. Check your virtual inbox: inspect the new email titled 'New Todo Created: Complete Lab 6'. View the HTML tab to verify the rendered layout, title and badge.
7. Use Postman with Basic Auth ('admin' / 'admin123') to send POST http://localhost:8080/api/v1/todos with JSON body: {"title": "REST email test", "completed": false}.
8. Verify that Postman receives HTTP 200 OK instantly and another HTML email arrives in your Mailpit inbox.

✓ Step 9. Summary

- The pom.xml file includes spring-boot-starter-mail dependency.
- application.properties correctly specifies SMTP host, port, credentials and TLS settings.
- TodolistApplication.java is decorated with @EnableAsync to allow sending email asynchronously.
- The templates/mail/todo-notification.html file renders dynamic todo properties with Thymeleaf.
- EmailService uses JavaMailSender and TemplateEngine inside an @Async method.
- TodoService calls EmailService upon todo creation.
- Submitting the HTML form or REST API creates the todo in MySQL and delivers an HTML email with zero UI delay.